

Contents

TO-BE Semi Detallado Ajustado	3
1. Resumen Ejecutivo	3
1.1 Contexto	3
Claridad organizacional sobre apoyos transversales	3
1.2 Decisión ejecutiva	4
1.3 Qué ya está en ejecución	4
1.4 Qué tranquilidad brinda el modelo	5
1.5 Mensaje para la organización	5
1.6 Próximo paso	6
2. Alcance del TO-BE y Base Documental	6
2.1 Alcance del documento	6
2.2 Qué no reemplaza este documento	7
2.3 Relación entre AS-IS, TO-BE, guías y transición	7
2.4 Base documental del modelo TO-BE	8
2.5 Decisiones respaldadas por la documentación	10
Gobierno documental de DoD y clasificación Desarrollo / Soporte	10
2.6 Límites del documento	11
2.7 Lectura correcta del TO-BE	11
3. Modelo Objetivo por Capacidades	11
3.1 Enfoque general	11
3.2 Vista consolidada de capacidades TO-BE	12
3.3 Gobierno del trabajo	13
3.4 Arquitectura de integración y protección del Core Legacy	13
3.5 Reportería gobernada	13
3.6 Calidad funcional integrada	14
3.7 Calidad técnica automatizada	14
3.8 Gobierno de código y despliegues	14
3.9 Organización por capacidades y frentes	15
3.10 Capacidad y métricas	15
3.11 Transición controlada	15
4. Matriz de Respuesta a Hallazgos del AS-IS	15
4.1 Propósito de la matriz	15
4.2 Matriz consolidada AS-IS → TO-BE	16
4.3 Lectura ejecutiva de la respuesta TO-BE	17
5. Modelo Organizacional y Responsables	18
5.1 Propósito del modelo organizacional	18
Recomendación ejecutiva — Jefatura de Operación Tecnológica	18
5.2 Capacidades operativas principales	19
5.3 Frentes operacionales de Desarrollo y Arquitectura	19
5.4 Estructura operativa objetivo	20
5.5 Responsables por frente y apoyo de configuración Azure DevOps	21
5.6 Equipos identificados por frente	21
5.7 Rol de José Camilo Verdugo	22
5.8 Rol de Alexandra Castillo	22
5.9 Regla de usuarios mínimos por proyecto	23
5.10 Gobierno de demanda por líneas de negocio	23
5.11 Matriz de límites de responsabilidad	24
5.12 Gobierno de proveedores	24
5.13 Relación con Recursos Humanos	24
5.14 Resultado esperado del modelo organizacional	25
6. Impacto por Audiencia	25
6.1 Propósito	25
6.2 Matriz de impacto	25
6.3 Desarrollo y Arquitectura	26
6.4 QA / BA	26

6.5 Infraestructura / Cloud / Operaciones	27
6.6 SRE / Cloud / Sonar	27
6.7 Líneas de negocio	27
6.8 Gerencia General	28
6.9 Soporte	28
6.10 Recursos Humanos	28
6.11 Proveedores	29
7. Estado Actual, Brechas y Riesgos	29
7.1 Propósito	29
7.2 Estado ejecutivo	29
7.3 Qué ya se hizo	30
7.4 Qué está en ejecución	31
7.5 Qué falta formalizar	31
7.6 Riesgos principales	31
7.7 Brechas críticas por prioridad	32
7.8 Lectura ejecutiva del estado actual	32
8. Primera Ola de Adopción y Plan de Transición	32
8.1 Propósito del plan de transición	32
8.2 Principio rector de la transición	33
8.3 Hito operativo	33
8.4 Alcance de la primera ola	33
8.5 Secuencia de adopción inicial	33
8.6 Configuración mínima de cada proyecto	34
8.7 Reglas inmediatas	34
8.8 Decisiones pendientes	34
8.9 Entregables esperados del plan de transición	35
8.10 Criterios de éxito de la transición	35
8.11 Riesgos de transición y mitigación	35
8.12 Cierre ejecutivo	36

Ajuste complementario — Gobierno de demanda y refinamiento anticipado del backlog	36
A. Punto de partida ajustado	36
B. Regla TO-BE de refinamiento anticipado a 2-3 sprints	36
C. Objetivo del refinamiento anticipado	37
D. Flujo objetivo de preparación	37
E. Responsabilidades por rol	37
F. Regla de Sprint Planning	37
G. Resultado esperado	38

Ajuste complementario — Frentes operacionales, Jefferson, Alexandra y límites de responsabilidad	38
1. Claridad organizacional sobre Jefferson Contreras	38
2. Relación funcional entre Jefferson y Alexandra	38
3. Límites del rol de Alexandra Castillo	38
4. Jefferson y la calidad técnica del código	39
5. Matriz ajustada de responsabilidad	39
6. Lectura correcta del modelo	40
7. Riesgos que evita esta precisión	40

Ajuste complementario final — Primera ola de migración de HU y clasificación de demanda para jefes de línea	41
1. Propósito	41
2. Ruta real de adopción de la primera ola	41
3. Paso 3 corregido — Capacitación y carga asistida con Alexandra	41
4. Responsabilidad de los jefes de línea	41
5. Guía de clasificación para negocio	41
6. Regla de casos grises	42
7. Campos mínimos para carga inicial	42

8. Control de duplicidades durante migración	42
9. Estado de una HU cargada	43
10. Resultado esperado	43

TO-BE Semi Detallado Ajustado

1. Resumen Ejecutivo

1.1 Contexto

La organización ya inició la transición hacia un modelo TO-BE orientado a operar como una fábrica de software gobernada, trazable, auditable y escalable.

Este modelo nace del AS-IS previamente levantado y responde a una realidad concreta: la compañía tiene una operación tecnológica con alta demanda de desarrollo, soporte, reportes, integraciones, calidad, despliegues y atención a negocio. El reto no es únicamente construir más software, sino gobernar mejor cómo se solicita, se prioriza, se desarrolla, se valida, se despliega, se soporta y se mide.

El TO-BE no reemplaza las guías operativas ya construidas. Las articula como documento rector para explicar decisiones, responsabilidades, límites organizacionales y ruta de adopción.

Una precisión importante para la lectura de este documento es que la primera ola de adopción se concentra en la capacidad operativa de **Desarrollo y Arquitectura**, liderada por **Jonathan Moreno**. Esta capacidad se reorganiza en tres frentes operacionales dentro de Azure DevOps:

Desarrollo y Arquitectura

- Humano-SL
- Fomag-SL
- Paralelos-SL

Las capacidades de **QA / BA**, liderada por **Jenny Quintana**, y de **Infraestructura / Operaciones**, liderada por **Dayana González**, se articulan con el modelo, pero no son reorganizadas desde este documento. Esta delimitación es clave porque conserva los límites reales de responsabilidad: el frente Desarrollo y Arquitectura ajusta su operación, mientras QA/BA e Infraestructura se integran al flujo desde sus propias responsabilidades.

Claridad organizacional sobre apoyos transversales

Los apoyos transversales de SRE / Cloud, adopción Azure DevOps y reportería pertenecen organizacionalmente al área de **Desarrollo y Arquitectura**, bajo el liderazgo de **Jonathan Moreno**.

Su transversalidad es funcional, no jerárquica: apoyan a Humano-SL, Fomag-SL y Paralelos-SL según la necesidad del frente, pero no constituyen áreas independientes ni equipos paralelos.

Desarrollo y Arquitectura

Líder: Jonathan Moreno

- Humano-SL
 - Desarrollo
 - Líder: John López
 - Soporte
 - Líder: Iván Díaz
- Fomag-SL
 - Líder de Soporte y Desarrollo: David Suescun
- Paralelos-SL
 - Líder de Soporte y Desarrollo: Duvan Sarmiento
- Apoyos transversales del área
 - Jefferson Contreras: SRE / Cloud / Azure DevOps / CI-CD / Sonar / gobierno técnico
 - Alexandra Castillo: apoyo documental, organización de backlog, trazabilidad y adopción
 - José Camilo Verdugo: reportería y generación de reportes

1.2 Decisión ejecutiva

La decisión es evolucionar hacia una operación tecnológica con gobierno común para Desarrollo, Soporte, QA, BA, Infraestructura, negocio y proveedores, empezando por la adopción formal en los proyectos Azure DevOps de Humano-SL, Fomag-SL y Paralelos-SL.

Decisión TO-BE	Resultado esperado
Adoptar Azure DevOps bajo CMMI. Crear proyectos operativos para Humano-SL, Fomag-SL y Paralelos-SL. Tener dos tableros obligatorios por proyecto: Desarrollo y Soporte. Integrar QA y BA al flujo.	Trabajo visible, trazable y medible. Separar demanda, equipos, permisos, tableros y métricas por frente. Diferenciar trabajo planificado de operación, incidentes y bugs productivos. Reducir ambigüedad, reproceso y validaciones tardías.
Incorporar SonarQube, PR y Quality Gates.	Controlar calidad técnica antes de promover código.
Gobernar despliegues con CI/CD. Proteger el Core Legacy con APIs, ACL y reducción de accesos directos. Gobernar reportes por CódigoReporte.	Reducir pases manuales y mejorar trazabilidad. Disminuir riesgo técnico y acoplamiento.
Medir velocidad, WIP, backlog y capacidad. Formalizar una transición con hito de operación el 1 de julio de 2026.	Evitar SQL libre, vistas expuestas y consultas no controladas. Tomar decisiones con datos. Adoptar el modelo sin detener la operación.

1.3 Qué ya está en ejecución

El TO-BE no es una propuesta teórica. Ya existen avances concretos.

Frente	Estado
AS-IS TO-BE	Levantado y usado como insumo base. Definido en arquitectura, operación, calidad, gobierno y capacidad.
Base documental	Guías DevOps, CMMI, Work Items, Boards, BDD, ramas, PR, Sonar, CI/CD, versionamiento, Scrum, Kanban y onboarding documentadas.
Capacitación	Realizada inicialmente sobre el frente Desarrollo y Arquitectura.
Proveedores	Incluidos en la socialización inicial del modelo.
Organización Desarrollo y Arquitectura	Definida en tres frentes operacionales: Humano-SL, Fomag-SL y Paralelos-SL.
Reuniones de negocio	En curso con responsables de línea para levantar Historias de Usuario y bugs productivos.
Azure DevOps	Pendiente de creación/configuración formal de proyectos Humano-SL, Fomag-SL y Paralelos-SL.
Transición	Con hito operativo objetivo para el 1 de julio de 2026.

El estado actual puede resumirse así:

No se está empezando desde cero.

El AS-IS ya existe.

La base documental ya fue construida.

Desarrollo y Arquitectura ya fue capacitado inicialmente.

La transición ya inició con negocio.

Ahora se deben crear y operar los proyectos Azure DevOps bajo el estándar definido.

1.4 Qué tranquilidad brinda el modelo

El TO-BE brinda tranquilidad porque convierte problemas operativos en mecanismos de control.

Preocupación actual	Respuesta del TO-BE
No saber exactamente qué está haciendo cada equipo. No poder medir la capacidad real.	Trabajo visible en Azure DevOps, clasificado por proyecto, tablero, estado y responsable. Métricas de velocidad, WIP, backlog, tiempos de ciclo y carga por frente.
Tener responsables difusos.	Tres capacidades operativas principales y frentes definidos dentro de Desarrollo y Arquitectura.
Que no exista una lectura integral entre Desarrollo, QA / BA e Infraestructura.	La Jefatura de Operación Tecnológica, si es formalizada por Gerencia General, articularía las tres capacidades y respondería por gobierno, capacidad, riesgos, prioridades y resultados del flujo tecnológico completo.
Que QA llegue tarde.	QA integrado dentro de los tableros de Desarrollo y Soporte.
Que BA entregue requerimientos ambiguos.	Historias de Usuario con criterios verificables y escenarios BDD.
Que Infraestructura reciba pases manuales o poco trazables.	CI/CD, artefactos versionados, ambientes y aprobaciones.
Que producción reciba cambios riesgosos.	Pull Requests, Branch Policies, Build Validation, SonarQube y Quality Gates.
Que el Core Legacy quede expuesto.	ACL, APIs controladas y eliminación progresiva de accesos directos a datos.
Que los reportes crezcan sin gobierno.	API de reportes basada en CódigoReporte, catálogo y filtros permitidos.
Que la necesidad de más personal sea subjetiva.	Solicitudes de capacidad soportadas en datos de demanda, velocidad y backlog.

La tranquilidad no consiste en prometer que todo será más rápido de inmediato. Consiste en que la organización podrá saber qué se puede atender, qué está bloqueado, qué requiere priorización, qué implica riesgo y qué capacidad real existe.

1.5 Mensaje para la organización

El mensaje central del TO-BE es:

Estamos pasando de una operación reactiva, fragmentada y difícil de medir, a una fábrica de software gobernada, trazable, auditable y escalable.

Este cambio no busca imponer burocracia. Busca ordenar la realidad actual de la compañía.

No se trata de...	Se trata de...
Hacer más reuniones.	Hacer visible el trabajo.
Frenar al negocio.	Priorizar con claridad.
Cargar más a QA.	Integrar la calidad desde el inicio.
Complicar a Desarrollo.	Proteger el código y reducir reproceso.
Hacer que Infraestructura sea un cuello de botella.	Darle trazabilidad y control sobre despliegues.
Pedir más personas por percepción.	Sustentar la capacidad con datos.
Cambiar todo de golpe.	Transicionar de forma gradual y controlada.

La empresa ya funciona como una fábrica de software. El TO-BE permite gobernarla como tal.

1.6 Próximo paso

El siguiente paso es ejecutar la primera ola de adopción mediante la configuración de los proyectos Azure DevOps:

Humano-SL
Fomag-SL
Paralelos-SL

Cada proyecto deberá tener, como mínimo:

- Board Desarrollo.
- Board Soporte.
- Usuarios reales del frente.
- Roles y permisos mínimos.
- Backlog inicial de HU y bugs productivos.
- Reglas de Work Items, ramas, PR, Sonar, CI/CD y evidencias.

A partir del **1 de julio de 2026**, las nuevas Historias de Usuario y bugs productivos deberán gestionarse dentro del proyecto y tablero correspondiente en Azure DevOps.

2. Alcance del TO-BE y Base Documental

2.1 Alcance del documento

Este documento consolida el modelo TO-BE en ejecución para la evolución de la operación tecnológica hacia una fábrica de software gobernada, trazable, auditable y escalable.

Su alcance es presentar, de forma ejecutiva y arquitectónica, las decisiones tomadas, las capacidades objetivo, los responsables principales, el impacto por audiencia y los elementos pendientes para formalizar la transición.

El documento cubre los siguientes frentes:

Frente	Alcance dentro del TO-BE
Arquitectura empresarial	Protección del Core Legacy, reducción de accesos directos a datos, integración controlada y evolución progresiva.
Gobierno operativo	Uso de Azure DevOps bajo CMMI para trazabilidad, tableros, Work Items, evidencias y medición.
Organización	Separación entre capacidades operativas principales y frentes operacionales de Desarrollo y Arquitectura.
Calidad funcional	Participación de BA y QA desde la definición de historias, criterios de aceptación y escenarios BDD.
Calidad técnica	Uso de SonarQube, Quality Gates, revisión técnica, estándares de código y control de deuda técnica.
DevOps / CI-CD	Gobierno de ramas, Pull Requests, pipelines, artefactos versionados, ambientes y despliegues trazables.
Reportes	Nueva API de reportes gobernada por CodigoReporte, catálogo, vistas registradas y filtros permitidos.
Soporte y operación	Flujo Kanban / ITSM liviano para incidentes, triage, análisis, escalamiento y cierre con evidencia.

Frente	Alcance dentro del TO-BE
Capacidad	Medición de velocidad, WIP, backlog, tiempos de ciclo, calidad y necesidad de refuerzo.
Transición	Ruta para adoptar el modelo desde la primera ola Humano-SL, Fomag-SL y Paralelos-SL.

Este documento no debe entenderse como un manual de configuración. Su objetivo es explicar el modelo objetivo y orientar la adopción organizacional.

2.2 Qué no reemplaza este documento

El TO-BE no reemplaza las guías operativas ya construidas y socializadas.

Las guías siguen siendo la fuente de detalle para configurar, operar y auditar cada práctica específica. Este documento las articula, las conecta y las traduce en decisiones empresariales.

Este documento TO-BE	Guías operativas
Explica el modelo objetivo. Consolida decisiones.	Explican cómo ejecutar cada práctica. Detallan configuraciones, pasos y reglas específicas.
Traduce impacto organizacional. Sirve para audiencias ejecutivas y transversales.	Describen procedimientos técnicos y operativos. Sirven para líderes, equipos técnicos, QA, BA, SRE e Infraestructura.
Define capacidades, límites y dirección. Identifica brechas y próximos pasos.	Definen implementación, checklists y evidencia. Estandarizan la operación diaria.

2.3 Relación entre AS-IS, TO-BE, guías y transición

El modelo documental se entiende como una cadena lógica.

AS-IS



Identifica hallazgos, riesgos y oportunidades de mejora.

TO-BE



Define el modelo objetivo, decisiones, capacidades y responsables.

Guías operativas



Explican cómo ejecutar el modelo en la práctica.

Plan de transición



Define cómo pasar ordenadamente del estado actual al modelo objetivo.

Entregable	Rol dentro del cambio	Estado
AS-IS	Diagnóstico base de la situación actual.	Levantado.
TO-BE	Modelo objetivo y respuesta a los hallazgos.	En ejecución inicial.
Guías operativas	Estándares y procedimientos para operar el modelo.	Documentadas y socializadas inicialmente.

Entregable	Rol dentro del cambio	Estado
Onboarding / checklists SRE	Mecanismo para habilitar proyectos y repositorios bajo el estándar.	Documentado.
Piloto / proyectos de referencia	Laboratorio de adopción controlada y recolección de evidencias.	Documentado como referencia.
Plan de transición	Ruta formal para adopción gradual y transversal.	En construcción.

2.4 Base documental del modelo TO-BE

La base documental existente se organiza en cinco bloques.

2.4.1 Bloque 1 — Diagnóstico AS-IS

Documento / insumo	Función dentro del TO-BE
AS-IS de soporte lógico	Identifica problemas operativos, técnicos, organizacionales y de trazabilidad.
AS-IS final	Consolida hallazgos y sirve como base para la respuesta TO-BE.

Uso dentro del TO-BE:

- No se reescribe.
- No se repite en detalle.
- Se usa para construir la matriz de respuesta AS-IS → TO-BE.
- Se conserva como respaldo del diagnóstico.

2.4.2 Bloque 2 — Estándar Operativo DevOps / CMMI Este bloque define la forma estándar de operar la fábrica de software.

Documento	Función dentro del TO-BE
01_Introduccion_Modelo.md	Define el estándar base, reglas maestras y orden de lectura del modelo.
02_Gobierno_Equipos_Roles.md	Define estructura de equipos, roles, accesos y permisos mínimos.
03_Estrategia_WorkItems_Boards.md	Define tableros Desarrollo/Soporte, estados, WIP, DoR y evidencias.
04_Gestion_Backlog_BDD.md	Define cómo BA estructura historias, criterios y escenarios verificables.
05_Estrategia_Ramas_PR.md	Define Git Flow, ramas temporales, commits y Pull Requests.
06_Gobierno_Promocion_Seguridad.md	Define Branch Security, Branch Policies, aprobaciones y hotfix.
07_Calidad_Validacion_Sonar.md	Define SonarQube, Quality Gates, deuda técnica y protocolo ante fallos.
08_Estrategia_Pipelines_Ambientes.md	Define CI/CD, ambientes, artefactos, variables, secretos y aprobaciones.
09_Estandar_Versionamiento_Tecnico.md	Define congruencia entre Git, código fuente y artefacto.
10_Ejecucion_Desarrollo_Scrum.md	Define operación diaria de Desarrollo bajo Scrum adaptado.

Documento	Función dentro del TO-BE
11_Ejecucion_Soporte_Kanban.md	Define operación diaria de Soporte bajo Kanban / ITSM liviano.

Uso dentro del TO-BE:

- Soporta el modelo operativo.
- No se copia completo.
- Se referencia como estándar de ejecución.
- Permite demostrar que el TO-BE ya tiene disciplina documentada.

2.4.3 Bloque 3 — Onboarding de Proyecto Real

Documento	Función dentro del TO-BE
00_Guia_Uso_Onboarding_Proyecto_Real.md	Explica cuándo usar checklist de proyecto nuevo o repositorio nuevo.
01_Checklist_SRE_Onboarding_Proyecto_Nuevo.md	Define condiciones mínimas para crear un proyecto Azure DevOps gobernado.
02_Checklist_SRE_Onboarding_Repositorio_Nuevo.md	Define condiciones mínimas para habilitar repositorios, ramas, políticas, Sonar y CI/CD.

Uso dentro del TO-BE:

- Soporta la adopción controlada.
- Evita que nuevos proyectos nazcan sin gobierno.
- Permite estandarizar ambientes, permisos, repositorios y evidencias.
- Será insumo directo para Humano-SL, Fomag-SL y Paralelos-SL.

2.4.4 Bloque 4 — Implementación piloto / referencia Este bloque demuestra cómo aplicar el estándar en un entorno controlado.

Documento / insumo	Función dentro del TO-BE
Proyecto de referencia / piloto	Sirve como evidencia práctica para probar el modelo antes de escalarlo.
Runbooks de evidencias	Definen simulación de HU, incidente, release y evidencia de cierre.
Backlog de referencia	Define Work Items de referencia para validar el modelo.

2.4.5 Bloque 5 — Arquitectura técnica específica

Documento / insumo	Función dentro del TO-BE
MVP API Reportes .NET 9	Define arquitectura de reportes gobernada por CodigoReporte.
Análisis DiasDescansoRemunerado / CALENDARIODET_TH	Identifica cuello de botella y plantea refactorización para reducir consultas repetitivas.
Enfoque ACL / capa anticorrupción	Define protección del Core Legacy y reducción de accesos directos a datos.
Separación Humano-SL, Fomag-SL y Paralelos-SL	Define organización técnica y operativa por frentes dentro de Desarrollo y Arquitectura.

2.5 Decisiones respaldadas por la documentación

Decisión TO-BE	Documento soporte	Lectura ejecutiva
Operar bajo Azure DevOps CMMI. Separar Desarrollo y Soporte.	Guías de roles, Work Items, tableros, Scrum y Soporte. Guías de Boards, Scrum y Kanban / ITSM.	La operación será trazable y medible. Se diferencian demanda planificada e incidentes.
No crear tablero QA aislado. Fortalecer BA con BDD.	Guías de roles y tableros. Guía de backlog y redacción funcional.	QA se integra al flujo. Las historias serán verificables.
Controlar código con ramas y PR. Usar SonarQube como control técnico. Automatizar despliegues.	Guías de repositorios y promoción. Guía de calidad y Sonar. Guía de pipelines y ambientes.	No habrá cambios sin revisión y trazabilidad. Se controla deuda técnica y riesgo. Los despliegues serán auditables.
Versionar artefactos.	Estándar de versionamiento.	Se sabrá qué versión llegó a cada ambiente.
Usar checklists SRE.	Onboarding de proyecto y repositorio.	Nuevos componentes nacen gobernados.
Gobernar reportes.	MVP API Reportes.	Menos SQL libre y menos exposición de datos.
Proteger Core Legacy.	Enfoque ACL / integración controlada.	Menos acoplamiento y menor riesgo productivo.
Medir capacidad.	Modelo CMMI, tableros y métricas.	La necesidad de refuerzo se sustenta con datos.

Gobierno documental de DoD y clasificación Desarrollo / Soporte

El TO-BE no redefine en detalle la Definition of Done ni las reglas operativas de clasificación. Este documento conserva la decisión ejecutiva y remite el detalle a las guías de gobierno.

La regla ejecutiva es:

Ningún trabajo se considera terminado si no tiene trazabilidad, validación y evidencia.

La clasificación entre Desarrollo y Soporte se rige por la naturaleza de la demanda, no por el solicitante ni por la persona que la atiende.

Desarrollo: trabajo planificado, evolutivo o de construcción de producto.

Soporte: incidentes, bugs productivos, diagnóstico, estabilización y operación.

El detalle operativo se gobierna en:

Tema	Documento de gobierno
Tableros, estados, Work Items, DoR, DoD y evidencia de cierre.	03_Estrategia_WorkItems_Boards.md
Calidad funcional de HU, criterios de aceptación y BDD.	04_Gestion_Backlog_BDD.md
Ramas, commits, Pull Requests y trazabilidad técnica.	05_Estrategia_Ramas_PR.md
Promoción, seguridad de ramas, aprobaciones y hotfix.	06_Gobierno_Promocion_Seguridad.md
SonarQube, Quality Gates, deuda técnica y excepciones.	07_Calidad_Validacion_Sonar.md
CI/CD, ambientes, artefactos, Release Request y rollback.	08_Estrategia_Pipelines_Ambientes.md

Tema	Documento de gobierno
Cierre de Sprint y ejecución diaria de Desarrollo.	10_Ejecucion_Desarrollo_Scrum.md
Cierre operativo de Soporte, triage y escalamiento.	11_Ejecucion_Soporte_Kanban.md

2.6 Límites del documento

Tema	Tratamiento en este documento
Configuración detallada de Azure DevOps	Se referencia, no se repite.
Rutas de menú y pantallas	Permanecen en guías y checklists.
Código fuente o implementación técnica	No se incluye.
Cronograma detallado de transición	Queda para el plan de transición.
Matriz RACI definitiva	Se propone estructura, pero debe validarse con líderes.
Backlog priorizado real	Se define como pendiente de transición.
Métricas reales de capacidad	Se definen como objetivo; requieren tablero y datos consistentes.
Implementación completa de SonarQube	Se define como capacidad objetivo; debe aterrizar por repositorio.
Ejecución de la API de reportes	Se reconoce como frente en marcha; requiere backlog y plan técnico específico.

2.7 Lectura correcta del TO-BE

El TO-BE debe leerse como un documento de dirección y alineación.

No debe leerse como:

- Un manual de Azure DevOps.
- Una guía de Scrum.
- Un instructivo de SonarQube.
- Un runbook de pipelines.
- Un documento de código.
- Un cronograma cerrado.

Debe leerse como:

- La respuesta organizacional al AS-IS.
- El modelo objetivo de operación.
- La articulación de las guías ya construidas.
- La base para explicar el cambio a QA/BA, Infraestructura, negocio y gerencia.
- El insumo principal para construir el plan de transición.

3. Modelo Objetivo por Capacidades

3.1 Enfoque general

El modelo TO-BE se estructura por capacidades empresariales y operativas. Esto permite explicar el cambio de manera clara, sin convertir el documento en una guía operativa ni repetir el detalle técnico que ya existe en los documentos de gobierno.

Una capacidad representa una habilidad que la organización debe desarrollar o fortalecer para operar como una fábrica de software gobernada, trazable, auditable y escalable.

El TO-BE no se basa únicamente en herramientas. Se basa en un conjunto integrado de capacidades:

Capacidad	Propósito
Gobierno del trabajo Arquitectura de integración Reportería gobernada	Hacer visible, medible y trazable la demanda. Proteger el Core Legacy y reducir acoplamientos. Controlar generación de reportes y acceso a datos.
Calidad funcional	Mejorar la claridad de requerimientos y validaciones.
Calidad técnica	Reducir deuda, vulnerabilidades y defectos técnicos.
Gobierno de código y despliegues	Controlar cambios desde desarrollo hasta producción.
Organización por capacidades y frentes	Definir límites de responsabilidad y medición por equipo.
Capacidad y métricas Transición controlada	Tomar decisiones con datos reales. Adoptar el modelo sin detener la operación.

3.2 Vista consolidada de capacidades TO-BE

Capacidad TO-BE	Decisión objetivo	Qué cambia	Beneficio esperado	Estado actual
Gobierno del trabajo	Adoptar Azure DevOps bajo CMMI como fuente de trazabilidad.	El trabajo deja de depender de chats, correos o acuerdos informales.	Visibilidad de demanda, responsables, estados, bloqueos y evidencia.	Definido y documentado.
Arquitectura de integración	Implementar integración controlada mediante APIs, eventos y ACL.	Se reducen accesos directos a datos e integraciones espagueti.	Protección del Core Legacy y menor riesgo productivo.	Definido como orientación arquitectónica.
Reportería gobernada	Construir una API de reportes basada en CódigoReporte.	El cliente deja de enviar SQL, vistas o filtros libres.	Reportes más seguros, trazables y mantenibles.	Definido y con apoyo especializado asignado.
Calidad funcional	Integrar BA, QA y negocio mediante historias, criterios y escenarios BDD.	QA deja de operar como fase final aislada.	Menos ambigüedad, menos reproceso y validaciones más objetivas.	Documentado; pendiente adopción transversal con QA/BA y negocio.
Calidad técnica	Incorporar SonarQube, Quality Gates y guía de codificación.	La calidad técnica deja de depender solo de revisión manual.	Menor deuda técnica, menos vulnerabilidades y mayor mantenibilidad.	Documentado; pendiente implementación progresiva por repositorio.
Gobierno de código	Usar ramas, Pull Requests, Work Items y revisión técnica.	Se elimina el cambio directo o sin trazabilidad sobre ramas permanentes.	Mayor control sobre lo que se integra y promueve.	Documentado y socializado con responsables técnicos.
Despliegues trazables	Adoptar CI/CD, ambientes, artefactos versionados y aprobaciones.	Infraestructura deja de recibir paquetes manuales o cambios poco auditables.	Despliegues repetibles, auditables y con menor riesgo.	Documentado; pendiente adopción progresiva.

Capacidad TO-BE	Decisión objetivo	Qué cambia	Beneficio esperado	Estado actual
Organización por capacidades y frentes	Separar Desarrollo y Arquitectura, QA/BA e Infraestructura; y dentro de Desarrollo operar Humano-SL, Fomag-SL y Paralelos-SL.	La responsabilidad deja de estar mezclada.	Mejor coordinación, medición y escalamiento.	En ejecución inicial.
Capacidad y métricas	Medir velocidad, WIP, backlog, tiempos de ciclo, calidad y carga.	La capacidad deja de ser una percepción.	Decisiones de priorización y contratación basadas en datos.	Definido; pendiente dashboard ejecutivo.
Transición controlada	Formalizar primera ola con hito del 1 de julio de 2026.	El TO-BE deja de depender de adopciones aisladas.	Cambio ordenado, gradual y sin detener la operación.	En ejecución inicial.

3.3 Gobierno del trabajo

Decisión TO-BE: Centralizar la gestión del trabajo técnico y operativo en Azure DevOps bajo el proceso CMMI.

Toda solicitud relevante debe quedar registrada, clasificada, priorizada y trazada. Esto aplica para desarrollo, soporte, bugs, incidentes, requerimientos, cambios, releases, despliegues y tareas técnicas.

Antes	TO-BE
Trabajo gestionado por chat, correo o acuerdos informales.	Trabajo visible mediante Work Items.
Dificultad para saber quién responde.	Responsable asignado por frente o Work Item.
Cierres sin evidencia homogénea.	Cierre soportado con evidencia.
Capacidad difícil de medir.	Métricas por equipo, frente y tipo de trabajo.

3.4 Arquitectura de integración y protección del Core Legacy

Decisión TO-BE: Proteger el Core Legacy mediante una arquitectura de integración controlada, apoyada en APIs, eventos y capa anticorrupción.

Las nuevas soluciones no deben aumentar el acoplamiento con el legado ni acceder libremente a bases de datos o estructuras internas. La integración debe pasar por contratos y puntos controlados.

Antes	TO-BE
Integraciones directas o difíciles de gobernar.	Integración mediante APIs, eventos y ACL.
Exposición del modelo interno del legacy.	Encapsulamiento y protección del Core Legacy.
Mayor riesgo de impacto colateral.	Menor acoplamiento y evolución controlada.
Dependencias difíciles de rastrear.	Contratos más claros y trazables.

3.5 Reportería gobernada

Decisión TO-BE: Implementar una API de reportes en .NET 9 donde el consumidor solicite reportes mediante CodigoReporte.

El consumidor no envía SQL, nombres de vistas ni filtros arbitrarios. La API valida el reporte contra un catálogo autorizado, aplica filtros permitidos y genera la salida correspondiente.

Antes	TO-BE
Reportes con riesgo de consultas no gobernadas. Posible exposición de vistas o SQL. Dificultad para controlar filtros y datos expuestos. Crecimiento desordenado de reportes.	Reportes registrados y controlados por catálogo. Consumo mediante CódigoReporte. Filtros permitidos y consultas parametrizadas. Gobierno especializado del componente de reportes.

Apoyo especializado: José Camilo Verdugo participa como apoyo especializado cuando el backlog de Humano-SL, Fomag-SL o Paralelos-SL incluya solicitudes, bugs o componentes relacionados con reportería.

3.6 Calidad funcional integrada

Decisión TO-BE: Integrar BA y QA desde el inicio del flujo mediante Historias de Usuario claras, criterios de aceptación verificables y escenarios BDD.

BA no solo recibe solicitudes: las estructura. QA no valida al final como silo: participa dentro del flujo de Desarrollo y Soporte.

Antes	TO-BE
Requerimientos ambiguos o incompletos. QA como etapa final. Validaciones subjetivas. Reproceso por mala definición.	Historias con contexto, valor y criterios claros. QA integrado al flujo. Validaciones contra criterios y escenarios. Menos ambigüedad desde el inicio.

3.7 Calidad técnica automatizada

Decisión TO-BE: Incorporar SonarQube, Quality Gates y guía de codificación como controles de calidad técnica.

SonarQube no reemplaza QA ni la revisión técnica; complementa el flujo con análisis automatizado de bugs, vulnerabilidades, duplicidad, mantenibilidad y deuda técnica.

Antes	TO-BE
Calidad técnica dependiente principalmente de criterio manual. Deuda técnica poco visible. Riesgo de promover vulnerabilidades o defectos críticos. Reglas de código no institucionalizadas.	Validación automatizada con SonarQube. Deuda técnica medible y gestionable. Quality Gates por ambiente. Guía de codificación integrada al gobierno técnico.

3.8 Gobierno de código y despliegues

Decisión TO-BE: Controlar el ciclo de vida del código mediante ramas, Pull Requests, Work Items, validaciones, pipelines y artefactos versionados.

El código debe avanzar por un flujo trazable desde la necesidad hasta el despliegue. Las promociones entre ambientes deben estar controladas y soportadas con evidencia.

Antes	TO-BE
Cambios con baja trazabilidad. Riesgo de push directo o promoción informal. Despliegues manuales o poco auditables. Dificultad para saber qué versión está desplegada.	Cambios asociados a Work Items. Pull Requests y políticas de rama. CI/CD y artefactos versionados. Versionamiento técnico y evidencia de release.

3.9 Organización por capacidades y frentes

Decisión TO-BE: Diferenciar capacidades organizacionales principales y frentes operacionales.

Tipo	Elementos
Capacidad organizacional	Desarrollo y Arquitectura, QA/BA, Infraestructura/Operaciones.
Frente operacional de Desarrollo y Arquitectura Apoyo especializado	Humano-SL, Fomag-SL, Paralelos-SL. Reportes, adopción Azure DevOps, SRE/Cloud/Sonar.

Esta distinción evita mezclar áreas que no tienen la misma autoridad, dependencia ni propósito operativo.

3.10 Capacidad y métricas

Decisión TO-BE: Medir capacidad mediante indicadores de demanda, velocidad, WIP, backlog, tiempos de ciclo, calidad, deuda técnica y soporte.

Antes	TO-BE
“Estamos muy cargados”, sin evidencia consolidada. Solicitud de personal basada en percepción. Dificultad para comparar frentes. Priorización reactiva.	Backlog, velocidad, WIP y tiempos de ciclo visibles. Solicitud de refuerzo basada en datos. Métricas comunes por equipo. Priorización según capacidad y valor.

3.11 Transición controlada

Decisión TO-BE: Formalizar una transición por fases, con primera ola de adopción sobre Humano-SL, Fomag-SL y Paralelos-SL.

El hito operativo es el **1 de julio de 2026**. A partir de esa fecha, las nuevas Historias de Usuario y bugs productivos deberán gestionarse en Azure DevOps dentro del proyecto y tablero correspondiente.

4. Matriz de Respuesta a Hallazgos del AS-IS

4.1 Propósito de la matriz

Esta sección conecta los hallazgos del AS-IS con las decisiones TO-BE ya definidas o en ejecución.

No se rehace el diagnóstico. El AS-IS se toma como insumo cerrado. La matriz permite mostrar, de forma práctica y ejecutiva, cómo responde el TO-BE a cada frente de mejora identificado.

4.2 Matriz consolidada AS-IS → TO-BE

Frente	Hallazgo AS-IS	Decisión TO-BE	Responsable / frente	Beneficio esperado	Estado
Gobierno y trazabilidad	Trabajo disperso, baja trazabilidad y dificultad para saber estado real de solicitudes, cambios y cierres.	Adoptar Azure DevOps bajo CMMI como fuente de trazabilidad operativa.	Líderes de Desarrollo, Soporte, SRE / Cloud, QA y BA.	Trabajo visible, trazable, medible y auditable.	Definido / en adopción.
Organización	Responsabilidades difusas entre desarrollo, soporte, QA, BA, infraestructura y negocio.	Separar capacidades principales y frentes operacionales.	Jonathan Moreno, Jenny Quintana, Dayana González y líderes de frente.	Claridad de responsables, escalamiento y seguimiento.	En ejecución inicial.
QA / BA	QA operando tarde en el flujo y requerimientos con ambigüedad funcional.	Integrar QA al flujo y fortalecer BA con historias basadas en escenarios BDD.	Jenny y equipo QA / BA.	Menos reproceso, mejor validación y criterios más objetivos.	Pendiente socialización transversal.
Infraestructura	Despliegues manuales, poco trazables o con dependencia de instrucciones informales.	Adoptar CI/CD, ambientes controlados, artefactos versionados y aprobaciones.	Dayana, Infraestructura, Jefferson, SRE / Cloud, CI/CD.	Despliegues más seguros, repetibles y auditables.	Definido / pendiente adopción progresiva.
Código y promoción	Riesgo de cambios sin PR, sin Work Item, sin revisión o saltando ambientes.	Adoptar flujo dev → sit → uat → staging → main, Branch Security, Branch Políticas y PR obligatorio.	Líder Desarrollo, Responsable CI, QA, Soporte.	Control de cambios y reducción de riesgo productivo.	Definido.
Calidad técnica	Calidad dependiente de revisión manual y deuda técnica poco visible.	Incorporar SonarQube, Quality Gates y guía de codificación.	Jefferson / Responsable Sonar, líderes técnicos, CI/CD.	Menor deuda técnica, menos vulnerabilidades y mejor mantenibilidad.	Definido / pendiente implementación por repositorio.
Reportes	Reportes acoplados a consultas, vistas o lógica difícil de gobernar.	Crear API de reportes en .NET 9 basada en CódigoReporte.	José Camilo Verdugo como apoyo especializado de reportes.	Menos SQL libre, más gobierno y trazabilidad de reportes.	Definido / en ejecución inicial.

Frente	Hallazgo AS-IS	Decisión TO-BE	Responsable / frente	Beneficio esperado	Estado
Integración / Core Legacy	Integraciones espagueti y accesos directos a bases de datos.	Proteger el Core Legacy mediante APIs, eventos y ACL / capa anticorrupción.	Arquitectura, Desarrollo, Infraestructura.	Menor acoplamiento, menor riesgo y evolución controlada.	Definido.
Performance / Legacy	Procesos críticos con consultas repetitivas y presión innecesaria sobre base de datos.	Refactorizar puntos críticos como DiasDescansoRemunerado / CALENDARI-ODET_TH.	Arquitectura y líderes técnicos del componente.	Mejor rendimiento, menos round-trips y menor presión sobre SQL Server.	Identificado / pendiente plan de implementación.
Soporte	Atención reactiva, trabajo invisible y escalamientos sin diagnóstico suficiente.	Adoptar Kanban / ITSM liviano con Triage, Analysis, evidencia y escalamiento controlado.	Líder Soporte y equipo de soporte por frente.	Menos interrupciones innecesarias a desarrollo y mejor control operativo.	Definido / en adopción.
Onboarding	Proyectos o repositorios pueden nacer sin gobierno común.	Aplicar checklists SRE y proyectos de referencia para adopción controlada.	Jefferson, SRE / Cloud y responsables técnicos.	Nuevos componentes nacen con gobierno, seguridad, pipelines y evidencia.	Documentado y socializado con responsables.
Capacidad	Sensación de saturación sin datos consolidados para justificar refuerzo.	Medir velocidad, WIP, backlog, tiempos de ciclo, calidad y carga por frente.	Gerencia técnica, líderes de frente, RRHH.	Decisiones de capacidad basadas en datos, no percepción.	Pendiente dashboard ejecutivo.

4.3 Lectura ejecutiva de la respuesta TO-BE

Eje	Antes / hallazgo	Ahora / respuesta TO-BE
Trabajo	Disperso y difícil de medir.	Azure DevOps CMMI, Work Items, tableros y evidencias.
Responsabilidad	Difusa o dependiente de personas.	Capacidades principales y frentes definidos.
QA / BA	Calidad tardía y requerimientos ambiguos.	QA integrado y BA con escenarios BDD.
Infraestructura	Pases manuales o poco trazables.	CI/CD, ambientes, artefactos versionados y aprobaciones.
Código	Cambios con riesgo de baja trazabilidad.	PR, Branch Policies, Work Items y validaciones.
Calidad técnica	Deuda y riesgos poco visibles.	SonarQube, Quality Gates y guía de código.

Eje	Antes / hallazgo	Ahora / respuesta TO-BE
Arquitectura	Integraciones espagueti y accesos directos.	APIs, eventos, ACL y protección del Core Legacy.
Reportes	SQL/vistas/filtros difíciles de gobernar.	API de reportes por CódigoReporte.
Legacy	Consultas repetitivas y presión a base de datos.	Estrategia set-based o caché en memoria para puntos críticos.
Soporte	Reactivo e invisible.	Kanban / ITSM, Triage, Analysis y evidencia.
Onboarding	Arranques sin gobierno uniforme.	Checklists SRE y proyectos de referencia.
Capacidad	Saturación percibida.	Métricas para decidir y justificar refuerzos.

5. Modelo Organizacional y Responsables

5.1 Propósito del modelo organizacional

El modelo organizacional TO-BE define cómo se ordena la operación por capacidades, frentes, responsables y roles transversales para que la compañía pueda operar como una fábrica de software gobernada, medible y escalable.

Este punto no pretende reemplazar organigramas internos, manuales de funciones ni matrices RACI detalladas. Su propósito es dejar claro quién responde por cada frente dentro del modelo objetivo y cómo se reduce la ambigüedad operativa.

El principio base es:

Todo frente debe tener responsable.
 Todo trabajo debe tener dueño.
 Toda validación debe tener evidencia.
 Toda decisión relevante debe poder trazarse.

Recomendación ejecutiva — Jefatura de Operación Tecnológica

Como evolución natural del modelo TO-BE, se recomienda evaluar la formalización de una **Jefatura de Operación Tecnológica** como rol de gobierno transversal sobre las tres capacidades operativas principales: Desarrollo y Arquitectura, QA / BA e Infraestructura / Operaciones.

La recomendación no busca crear una capa burocrática ni reemplazar liderazgos existentes. Su propósito es cerrar una brecha de gobierno: asegurar que el flujo tecnológico completo tenga una única lectura ejecutiva de capacidad, prioridad, riesgo, avance, calidad, despliegues, soporte y cumplimiento del modelo.

Responsabilidades esperadas:

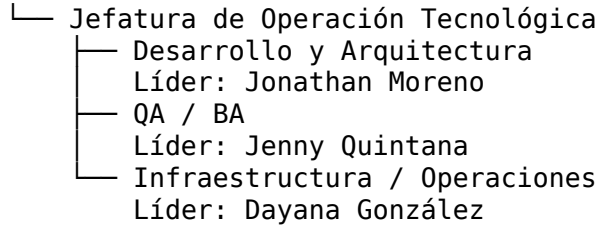
- Gobernar la operación tecnológica extremo a extremo.
- Priorizar conflictos entre Desarrollo y Arquitectura, QA / BA e Infraestructura / Operaciones.
- Responder ante Gerencia General por salud operativa, capacidad, riesgos y avance.
- Asegurar adopción real de Azure DevOps como fuente de verdad.
- Velar por métricas de backlog, WIP, velocidad, defectos, despliegues, disponibilidad y deuda.
- Resolver bloqueos entre áreas.
- Asegurar que las decisiones de negocio tengan lectura técnica, operativa y de capacidad.
- Cuidar que el modelo no se vuelva burocrático ni informal.

Límites explícitos del rol:

- No reemplaza al líder de Desarrollo y Arquitectura.
- No reemplaza a Jenny en el gobierno funcional de QA / BA.
- No reemplaza a Dayana en la operación de Infraestructura / Operaciones.
- No aprueba microdetalles operativos del día a día.
- No debe convertirse en cuello de botella de despliegues, Pull Requests, QA o soporte.

Estructura recomendada, sujeta a validación de Gerencia General:

Gerencia General



Esta figura permitiría que las tres capacidades conserven su especialidad, pero trabajen bajo una lectura integrada de operación, riesgo y capacidad.

5.2 Capacidades operativas principales

La lectura correcta del TO-BE parte de tres capacidades operativas principales. No deben mezclarse como si fueran frentes equivalentes, porque tienen liderazgos, responsabilidades y límites organizacionales diferentes.

Capacidad operativa principal	Liderazgo identificado	Alcance dentro del TO-BE	Tipo de intervención
Desarrollo y Arquitectura	Jonathan Moreno	Organiza y gobierna la ejecución de desarrollo, soporte técnico, arquitectura, reportes y adopción inicial de Azure DevOps para Humano-SL, Fomag-SL y Paralelos-SL.	Intervención directa del presente TO-BE.
QA / BA	Jenny Quintana	Asegura calidad funcional, historias verificables, criterios de aceptación, escenarios BDD y validaciones QA.	Articulación transversal; no se modifica su organigrama desde este documento.
Infraestructura / Operaciones	Dayana González	Habilita ambientes, despliegues, secretos, conexiones, continuidad operativa y soporte a CI/CD.	Articulación transversal; no se modifica su organigrama desde este documento.

Esta separación permite entender que la reorganización directa ocurre en **Desarrollo y Arquitectura**, mientras QA/BA e Infraestructura/Operaciones se integran como capacidades necesarias para que el flujo funcione.

5.3 Frentes operacionales de Desarrollo y Arquitectura

La capacidad operativa **Desarrollo y Arquitectura**, liderada por Jonathan Moreno, se divide en tres frentes operacionales:

Frente operacional	Propósito	Condición obligatoria
Humano-SL	Gestionar desarrollo, soporte, estabilización y evolución del frente Humano.	Debe tener Board Desarrollo y Board Soporte.
Fomag-SL	Gestionar desarrollo, soporte, estabilización y evolución del frente Fomag.	Debe tener Board Desarrollo y Board Soporte.
Paralelos-SL	Gestionar desarrollo, soporte, estabilización y evolución de otros frentes (ej: Efinomina).	Debe tener Board Desarrollo y Board Soporte.

Regla principal:

Todo proyecto operativo Azure DevOps de esta primera ola debe tener dos tableros:

1. Board Desarrollo.
2. Board Soporte.

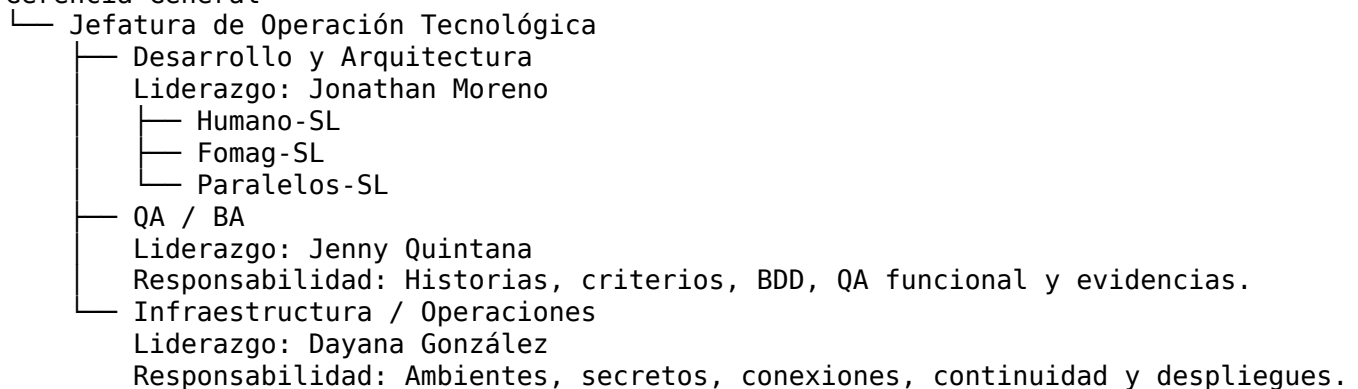
Cuando un proyecto tenga un solo líder, ese líder deberá coordinar ambas capacidades operativas: Desarrollo y Soporte.

5.4 Estructura operativa objetivo

La estructura operativa objetivo debe leerse así:

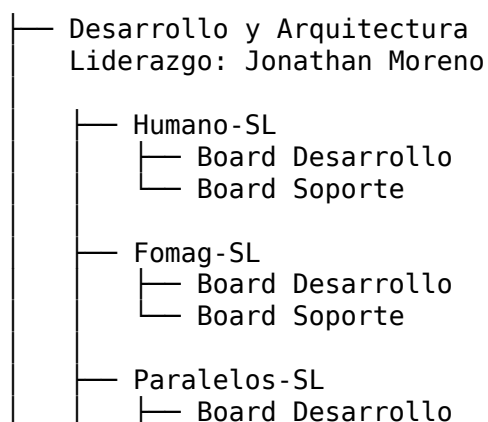
Vista recomendada si Gerencia General formaliza la Jefatura de Operación Tecnológica:

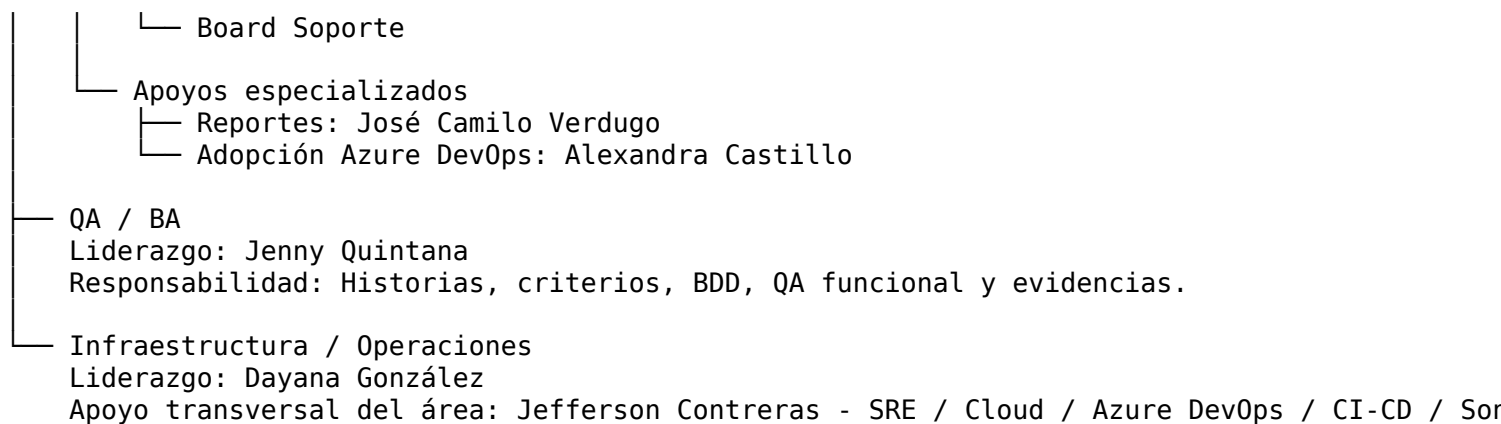
Gerencia General



Mientras esta jefatura no se formalice, la estructura debe seguir leyéndose por capacidades principales y coordinación entre líderes.

Compañía





Esta estructura no busca crear silos. Busca que cada capacidad tenga límites claros y que los frentes operacionales de Desarrollo y Arquitectura puedan medirse, priorizarse y gobernarse.

5.5 Responsables por frente y apoyo de configuración Azure DevOps

La configuración inicial de los proyectos Azure DevOps no debe ser una actividad aislada de SRE. Debe realizarse entre Jefferson, en rol SRE / Cloud, y el líder o líderes del frente correspondiente.

Proyecto Azure DevOps	Participantes obligatorios de configuración	Apoyos especializados
Humano-SL	John, líder Desarrollo; Iván, líder Soporte; Jefferson, SRE / Cloud.	Alexandra Castillo, apoyo operativo de adopción Azure DevOps; José Camilo Verdugo, apoyo de reportes cuando aplique.
Fomag-SL	David Suescun, líder del frente; Jefferson, SRE / Cloud.	Alexandra Castillo, apoyo operativo de adopción Azure DevOps; José Camilo Verdugo, apoyo de reportes cuando aplique.
Paralelos-SL	Duvan, líder del frente; Jefferson, SRE / Cloud.	Alexandra Castillo, apoyo operativo de adopción Azure DevOps; José Camilo Verdugo, apoyo de reportes cuando aplique.

Objetivo de estas sesiones:

- Crear o ajustar el proyecto Azure DevOps.
- Definir Team Desarrollo y Team Soporte.
- Validar que no exista Team QA ni Board QA independiente.
- Configurar usuarios, roles y permisos mínimos.
- Crear Board Desarrollo y Board Soporte.
- Definir Area Paths e Iteration Paths.
- Preparar backlog inicial.
- Identificar repositorios y componentes.
- Preparar reglas de ramas, PR, Sonar, pipelines y evidencias.

5.6 Equipos identificados por frente

Frente	Personas asociadas	Observación
Humano-SL — Desarrollo	John Lopez, Wilmer Sarmiento, Santiago González, Yurani.	Frente de construcción y evolución para Humano.
Humano-SL — Soporte	Iván Díaz, Víctor Medina, Duvan David Salas.	Frente operativo de soporte para Humano.
Fomag-SL	David Suescun.	Responsable general del frente; debe articular Desarrollo y Soporte.
Paralelos-SL	Duval, Diego Hurtado, Juan Moreno.	Frente de componentes y soluciones paralelas.
Reportes	José Camilo Verdugo.	Apoyo especializado cuando el backlog incluya reportería.
Adopción Azure DevOps	Alexandra Castillo.	Apoyo operativo en carga, organización y revisión mínima de HU y bugs.
QA / BA	Jenny Quintana y equipo QA / BA.	Frente funcional para historias, criterios, validación y evidencias.
SRE / Cloud / Sonar	Jefferson Contreras.	Apoyo transversal para configuración, SonarQube, pipelines y gobierno técnico.
Infraestructura / Operaciones	Dayana González y equipo de Infraestructura.	Ambientes, secretos, conexiones, continuidad y despliegues.

Esta información deberá validarse y complementarse en la matriz formal de transición, especialmente si cambian asignaciones, dedicaciones o prioridades.

5.7 Rol de José Camilo Verdugo

José Camilo Verdugo no debe quedar como un frente organizacional equivalente a Humano-SL, Fomag-SL o Paralelos-SL. Su rol debe entenderse como **apoyo especializado transversal de reportes** dentro de Desarrollo y Arquitectura.

Responsabilidades esperadas:

- Apoyar solicitudes relacionadas con reportes.
- Acompañar backlog de reportería cuando aplique.
- Gobernar criterios de catálogo, CódigoReporte, vistas registradas y filtros permitidos.
- Evitar SQL libre o consultas no gobernadas.
- Coordinar con QA y líderes de frente las validaciones cuando el cambio afecte reportes.

Regla:

José Camilo Verdugo participa cuando el backlog de Humano-SL, Fomag-SL o Paralelos-SL incluya

5.8 Rol de Alexandra Castillo

Alexandra Castillo, practicante SENA, apoyará desde Desarrollo y Arquitectura la adopción operativa en Azure DevOps.

Su rol recomendado es **apoyo documental y operativo de adopción Azure DevOps**, articulado con Jefferson y con los líderes de frente, siempre dentro del área de Desarrollo y Arquitectura.

Responsabilidades esperadas:

- Apoyar la carga inicial de Historias de Usuario y bugs productivos.
- Acompañar a responsables de negocio en la correcta ubicación de la demanda.
- Revisar completitud mínima de campos básicos.

- Apoyar organización inicial del backlog.
- Ayudar a diferenciar si el elemento debe ir a Desarrollo o Soporte.
- Servir como punto operativo de articulación entre Desarrollo y Arquitectura, SRE y responsabilidad

Límite del rol:

Alexandra no reemplaza a BA.

Alexandra no asume responsabilidad funcional sobre las historias.

Alexandra no define criterios de aceptación de negocio.

Alexandra apoya la adopción operativa y la correcta carga de información en Azure DevOps.

Esta precisión evita invadir la responsabilidad funcional del área QA / BA liderada por Jenny.

5.9 Regla de usuarios mínimos por proyecto

Cada proyecto Azure DevOps solo debe incluir los usuarios que realmente participan en ese frente, con los roles y permisos requeridos según su responsabilidad.

Regla:

No se deben cargar usuarios masivamente ni por conveniencia.

Cada usuario debe tener justificación operativa, rol definido y permiso mínimo necesario.

Beneficios:

- Mejor trazabilidad.
- Métricas más limpias.
- Menor ruido operativo.
- Menor exposición de información.
- Responsabilidad más clara por proyecto.
- Mejor control de permisos.

5.10 Gobierno de demanda por líneas de negocio

Los responsables de línea de negocio tienen una responsabilidad activa en la transición. No solo participan en reuniones; deben velar porque sus Historias de Usuario y bugs productivos queden registrados en el proyecto y tablero correspondiente.

Línea / frente	Responsable identificado
MEN administrativo	Edwin Gaona
MEN Secretarías de Educación Soporte Funcional	Mayerline Pineda
MEN Secretarías de Educación Soporte L2	Dayana González
SED Bogotá	Johan Miranda
FOMAG	María Victoria del Valle
Rama Judicial - Efinómina	Judith Gómez
Rama Judicial - Carrera	Ana María
Otros clientes públicos / privados	Edwin Gaona
Servicios especializados	Strey Vergara

Responsabilidad esperada de cada responsable funcional:

- Identificar la demanda de su línea.
- Priorizar la demanda inicial.
- Velar porque las Historias de Usuario estén en el proyecto correcto.
- Velar porque los bugs productivos estén en el tablero de Soporte correcto.
- Participar en aclaraciones funcionales cuando BA, QA o Desarrollo lo requieran.
- Evitar que la demanda siga entrando por canales informales.

5.11 Matriz de límites de responsabilidad

Tema	Responsable primario	Apoyo	Límite
Organización de Desarrollo y Arquitectura	Jonathan Moreno	Líderes de frente	Aplica directamente a Humano-SL, Fomag-SL y Paralelos-SL.
Configuración de proyectos Azure DevOps	Jefferson Contreras	Líderes de frente / Alexandra	No se configura sin claridad de responsables, usuarios, tableros y evidencia.
Historias de Usuario	Línea de negocio / BA	Alexandra como apoyo operativo	Alexandra apoya carga; no define funcionalmente.
Bugs productivos	Líder Soporte del frente	QA / Desarrollo	Deben ir al Board Soporte correspondiente.
Reportes	José Camilo Verdugo	Desarrollo / QA / Negocio	Aplica cuando el backlog incluya reportaría.
QA funcional	Jenny Quintana	QA / BA / Desarrollo	QA se integra al flujo; no se crea Board QA.
Infraestructura	Dayana González	Jefferson / líderes técnicos	Ambientes, secretos, conexiones y continuidad.
Sonar / Cloud / CI-CD	Jefferson Contreras	Infraestructura / líderes técnicos	Apoyo técnico transversal, no frente de negocio.
Priorización ejecutiva	Gerencia / responsables funcionales	Líderes de frente	Se decide con datos de backlog, capacidad y riesgo.

5.12 Gobierno de proveedores

Los proveedores hacen parte de la operación de la fábrica de software cuando participan en desarrollo, soporte, QA, infraestructura, reportes o componentes técnicos.

La decisión TO-BE es que no exista un flujo paralelo para proveedores.

Aspecto	Regla TO-BE
Asignación	Todo trabajo de proveedor debe estar asociado a Work Item.
Código	Todo cambio debe pasar por rama, PR y revisión.
Calidad	Debe cumplir estándares de código, Sonar y validación QA cuando aplique.
Evidencia	Toda entrega debe tener soporte verificable.
Prioridad	Debe alinearse con el líder del frente correspondiente.
Despliegue	No debe hacerse por fuera de pipelines, aprobaciones o flujo definido.
Medición	El trabajo debe poder compararse con equipos internos.

5.13 Relación con Recursos Humanos

El TO-BE reconoce que la compañía opera como una fábrica de software y que, para sostener el modelo, se requiere capacidad adecuada.

Ya se comunicó a Recursos Humanos el tipo de perfil recomendado para fortalecer los equipos.

Perfil recomendado	Justificación
Programadores con experiencia previa en soporte u operaciones.	Tienen mayor sensibilidad productiva, entienden incidentes, diagnostican con más criterio, aprenden más rápido el contexto y son más versátiles.

Esta decisión no reemplaza el análisis formal de capacidad. La solicitud de nuevos perfiles deberá sustentarse con métricas como backlog, velocidad, WIP, tiempos de ciclo, carga de soporte, defectos y deuda técnica.

5.14 Resultado esperado del modelo organizacional

Con este modelo organizacional, la compañía podrá responder con claridad:

Quién lidera Desarrollo y Arquitectura.
Quién lidera QA / BA.
Quién lidera Infraestructura / Operaciones.
Quién responde por Humano-SL.
Quién responde por Fomag-SL.
Quién responde por Paralelos-SL.
Quién apoya reportes.
Quién apoya adopción Azure DevOps.
Quién configura técnicamente con SRE.
Quién debe cargar y priorizar HU y bugs.
Quién debe dejar evidencia.

El resultado esperado es una operación con responsables visibles, mejor escalamiento, menos trabajo invisible y capacidad de medición por frente.

6. Impacto por Audiencia

6.1 Propósito

El TO-BE cambia la forma en que las áreas solicitan, construyen, validan, despliegan, soportan y miden el software.

Esta sección resume qué cambia para cada audiencia, qué regla debe adoptar y qué beneficio recibe.

6.2 Matriz de impacto

Audiencia	Cambio de chip	Nueva regla	Beneficio
Desarrollo y Arquitectura	La operación se divide en frentes medibles.	Humano-SL, Fomag-SL y Paralelos-SL operan con Board Desarrollo y Board Soporte.	Más foco, trazabilidad y capacidad medible.
QA / BA	La calidad no entra al final.	BA estructura historias verificables y QA valida dentro del flujo.	Menos reproceso y validación objetiva.
Infraestructura / Operaciones	Infraestructura no es receptor de pases manuales.	Los despliegues deben llegar con trazabilidad, artefacto y evidencia.	Menor riesgo operativo y mayor control.

Audiencia	Cambio de chip	Nueva regla	Beneficio
SRE / Cloud / Sonar	La configuración no se hace aislada.	SRE configura con líderes de frente y aplica checklists.	Proyectos nacen gobernados.
Líneas de negocio	La urgencia no puede saltarse el flujo.	Negocio vela por HU y bugs en el proyecto y tablero correcto.	Más previsibilidad y priorización real.
Gerencia General	No es un cambio técnico aislado.	El modelo debe respaldarse como estrategia de riesgo, capacidad y escalabilidad.	Decisiones con datos y menor riesgo productivo.
Soporte	No se escala sin diagnóstico.	Triage, Analysis y evidencia antes de escalar.	Menos reproceso y mejor control operativo.
Recursos Humanos	No basta contratar por tecnología.	Los perfiles deben responder a la realidad de una fábrica de software.	Mejor ajuste y menor curva de aprendizaje.
Proveedores	No deben operar con reglas paralelas.	Deben cumplir el mismo gobierno que los equipos internos.	Entregas comparables, auditables y trazables.

6.3 Desarrollo y Arquitectura

Aspecto	TO-BE
Responsable principal Qué cambia	Jonathan Moreno. Desarrollo y Arquitectura se organiza en tres frentes operacionales: Humano-SL, Fomag-SL y Paralelos-SL.
Nueva regla	Cada proyecto debe tener Board Desarrollo y Board Soporte.
Debe adoptar	Work Items, ramas, PR, Sonar, QA integrado, evidencia y métricas.
Debe dejar de pasar	Trabajo invisible, cambios sin Work Item, solicitudes por chat y cierres sin evidencia.

Mensaje clave:

Desarrollo y Arquitectura gana foco cuando la demanda entra clara, priorizada y trazable.

6.4 QA / BA

Aspecto	TO-BE
Responsable principal Qué cambia	Jenny Quintana. QA deja de operar como etapa aislada y BA fortalece la definición funcional desde el inicio.
Nueva regla	La calidad empieza en la Historia de Usuario.
Deben adoptar	Historias con criterios verificables, escenarios BDD, QA integrado y evidencia de validación.
Debe dejar de pasar	Historias ambiguas, validaciones tardías y cierres sin evidencia.

Mensaje clave:

BA estructura la necesidad.

QA valida el cumplimiento.

La calidad se integra al flujo, no se revisa solo al final.

6.5 Infraestructura / Cloud / Operaciones

Aspecto	TO-BE
Responsable principal Qué cambia	Dayana González, con apoyo SRE / Cloud. Infraestructura pasa de receptor de paquetes manuales a habilitador del flujo seguro.
Nueva regla Deben adoptar	No se despliega nada sin trazabilidad mínima. Ambientes controlados, secretos protegidos, Service Connections, pipelines y evidencias.
Debe dejar de pasar	Pases manuales oscuros, credenciales personales, despliegues sin artefacto y cambios sin evidencia.

Mensaje clave:

Infraestructura no debe resolver la improvisación.

Debe habilitar despliegues seguros, repetibles y auditables.

6.6 SRE / Cloud / Sonar

Aspecto	TO-BE
Responsable principal Qué cambia	Jefferson Contreras. SRE / Cloud / Sonar opera como apoyo técnico transversal para configurar proyectos, pipelines, Sonar y evidencias.
Nueva regla	No se inicia configuración sin claridad sobre alcance, responsables, usuarios, tableros, repositorios, ambientes y evidencia.
Debe adoptar	Checklists SRE, configuración conjunta con líderes de frente, permisos mínimos y trazabilidad técnica.
Debe dejar de pasar	Configuraciones aisladas sin validación de líderes o sin usuarios reales definidos.

Mensaje clave:

SRE habilita el gobierno técnico, pero la configuración debe hacerse con los líderes del frente

6.7 Líneas de negocio

Aspecto	TO-BE
Responsable principal Qué cambia	Responsables funcionales / PO. El negocio sigue definiendo necesidad y prioridad, pero debe hacerlo por el flujo establecido.
Nueva regla	Cada responsable de línea debe velar porque sus HU y bugs productivos estén en el proyecto y tablero correcto.

Aspecto	TO-BE
Deben adoptar	Solicitudes con contexto, valor, prioridad, impacto y criterios de aceptación. Pedidos por canales informales, cambios “para ayer” sin priorización y solicitudes técnicas sin análisis.
Debe dejar de pasar	

Mensaje clave:

El T0-BE no frena al negocio.

Ordena la demanda para que pueda priorizarse, estimarse y cumplirse mejor.

6.8 Gerencia General

Aspecto	TO-BE
Responsable principal	Diego Paredes / Santa Cruz.
Qué cambia	El TO-BE se gestiona como estrategia de riesgo, capacidad y escalabilidad.
Nueva regla	Gerencia decide con datos y respalda la disciplina operativa.
Debe adoptar	Seguimiento de backlog, velocidad, WIP, calidad, deuda técnica, soporte y capacidad.
Debe dejar de pasar	Decisiones de capacidad basadas solo en percepción o urgencias aisladas.

Mensaje clave:

La empresa ya opera como una fábrica de software.

El T0-BE permite gobernarla con datos, responsables y trazabilidad.

6.9 Soporte

Aspecto	TO-BE
Responsable principal	Líder de Soporte por frente.
Qué cambia	Soporte opera como frente visible y medible, no como canal informal de urgencias.
Nueva regla	No se escala a Desarrollo sin diagnóstico mínimo.
Deben adoptar	Triage, Analysis, evidencia, clasificación de casos y cierre documentado.
Debe dejar de pasar	Escalamientos incompletos, casos sin trazabilidad y trabajo invisible.

Mensaje clave:

Soporte diagnostica, evidencia y escala solo cuando corresponde.

6.10 Recursos Humanos

Aspecto	TO-BE
Responsable principal	Área de Recursos Humanos.

Aspecto	TO-BE
Qué cambia	La contratación debe alinearse con la realidad de una fábrica de software.
Nueva regla	Los perfiles deben responder a capacidad, operación, soporte y evolución del software.
Perfil recomendado	Programadores con experiencia previa en soporte u operaciones.
Beneficio	Mayor versatilidad, mejor criterio productivo y menor curva de aprendizaje.

Mensaje clave:

La necesidad de perfiles debe sustentarse con datos de backlog, velocidad, WIP y carga operativa.

6.11 Proveedores

Aspecto	TO-BE
Responsable principal	Proveedor + líder interno del frente.
Qué cambia	Los proveedores se integran al mismo gobierno interno.
Nueva regla	No debe existir un flujo paralelo para proveedores.
Deben adoptar	Work Items, ramas, PR, revisión, calidad, QA, evidencias y despliegues controlados.
Debe dejar de pasar	Entregas informales, cambios sin trazabilidad o evidencias incompletas.

Mensaje clave:

Todo proveedor debe ser medible y auditable bajo las mismas reglas internas.

7. Estado Actual, Brechas y Riesgos

7.1 Propósito

Esta sección muestra el estado real del TO-BE: qué ya está hecho, qué está en ejecución, qué falta formalizar y qué riesgos deben gestionarse.

La lectura correcta es:

El TO-BE ya está definido y en ejecución inicial.

El siguiente reto es consolidarlo mediante adopción transversal y primera ola de operación en

7.2 Estado ejecutivo

Dimensión	Estado actual	Lectura ejecutiva
AS-IS	Levantado.	El diagnóstico ya existe y no se rehace.
TO-BE	Definido en arquitectura, operación, calidad, gobierno y capacidad.	El modelo objetivo ya tiene decisiones concretas.

Dimensión	Estado actual	Lectura ejecutiva
Base documental	Construida y organizada.	Existen guías para ejecutar el estándar.
Capacitación	Realizada inicialmente sobre Desarrollo y Arquitectura.	La adopción profunda inició en el frente bajo responsabilidad directa.
Organización Desarrollo y Arquitectura	Frentes Humano-SL, Fomag-SL y Paralelos-SL definidos.	El cambio organizacional del área ya tiene estructura objetivo.
Reuniones con negocio	En curso.	Se está levantando la demanda inicial para Azure DevOps.
QA / BA	Pendiente alineación completa.	Jenny y su equipo deben adoptar BDD, criterios y QA integrado.
Infraestructura	Pendiente alineación completa.	Dayana y su equipo deben asumir el rol de habilitador CI/CD y operación segura.
SRE / Cloud / Sonar	Rol clave identificado.	Jefferson debe acompañar configuración de proyectos y repositorios.
Líneas de negocio	Socialización en ejecución.	Deben adoptar la nueva forma de solicitar, priorizar y validar.
Gerencia General	Pendiente presentación ejecutiva completa.	Debe respaldar el modelo como estrategia de riesgo y capacidad.
Plan de transición	En construcción.	Es el siguiente entregable crítico.

7.3 Qué ya se hizo

Frente	Avance
Diagnóstico	AS-IS levantado y usado como insumo base.
Modelo objetivo	TO-BE estructurado en arquitectura, operación, calidad, gobierno y capacidad.
Base documental	Guías operativas construidas para DevOps, CMMI, ramas, PR, Sonar, CI/CD, BDD, Scrum, Kanban y onboarding.
Desarrollo y Arquitectura	Capacitación inicial realizada con responsables del frente.
Proveedores	Incluidos en la socialización del modelo.
Organización	Definición de Humano-SL, Fomag-SL y Paralelos-SL como frentes operacionales.
Reportes	Gobierno de reportes identificado como apoyo especializado.
Calidad técnica	SonarQube definido como control técnico objetivo.
Código	Guía de escritura de código definida como base para revisión y calidad.
Arquitectura	Enfoque ACL / capa anticorrupción definido para proteger el Core Legacy.
Reportería	API de reportes por CódigoReporte definida.
Performance	CALENDARIODET_TH identificado como punto crítico a refactorizar.
Recursos Humanos	Perfil recomendado comunicado: programadores con experiencia previa en soporte u operaciones.

7.4 Qué está en ejecución

Frente	Actividad en curso
Negocio	Reuniones con responsables de línea para levantar HU y bugs productivos.
Desarrollo y Arquitectura	Preparación de la primera ola de Azure DevOps para Humano-SL, Fomag-SL y Paralelos-SL.
SRE / Cloud	Alineación para configurar proyectos bajo checklists SRE.
Backlog	Preparación de carga inicial de demanda por línea de negocio.
Adopción Azure DevOps	Definición de usuarios, roles y permisos por proyecto.
Reportes	Identificación de solicitudes o componentes de reportería dentro de los backlogs.

7.5 Qué falta formalizar

Brecha	Qué falta	Riesgo si no se atiende
Proyectos Azure DevOps	Crear Humano-SL, Fomag-SL y Paralelos-SL.	La adopción queda conceptual.
Tableros	Configurar Board Desarrollo y Board Soporte por proyecto.	Se mezcla demanda planificada con soporte.
Usuarios y permisos	Cargar solo usuarios reales del proyecto con permisos mínimos.	Ruido operativo, mala trazabilidad y exposición innecesaria.
Backlog inicial	Cargar HU y bugs productivos con responsables de negocio.	Inicio sin demanda visible.
QA / BA	Socialización con Jenny y equipo.	QA tardío, historias ambiguas y reproceso.
Infraestructura	Alineación con CI/CD, ambientes, secretos y evidencias.	Despliegues manuales o poco trazables.
Líneas de negocio	Explicar nueva forma de solicitar y priorizar.	Solicitudes informales y prioridades cruzadas.
Gerencia General	Presentar valor ejecutivo y decisiones requeridas.	El modelo puede verse como opcional o solo técnico.
SonarQube	Perfiles, umbrales, Quality Gates y excepciones.	Calidad técnica dependiente solo del criterio manual.
Métricas	Dashboard ejecutivo de capacidad, backlog, WIP y calidad.	Decisiones de capacidad basadas en percepción.

7.6 Riesgos principales

Riesgo	Impacto	Mitigación
Doble operación	Azure DevOps convive con chats, correos y reuniones como canales principales.	A partir del hito definido, no iniciar HU ni bugs productivos sin Work Item.
Mezcla de responsabilidades	QA/BA, Infraestructura, SRE, Reportes y frentes operacionales se interpretan como equivalentes.	Mantener la separación entre capacidades principales, frentes operacionales y apoyos especializados.

Riesgo	Impacto	Mitigación
Backlog mal cargado	Historias incompletas o bugs sin diagnóstico entran al tablero.	Responsables de negocio, BA, Alexandra y líderes de frente revisan completitud mínima.
Usuarios excesivos en proyectos	Métricas contaminadas y permisos innecesarios.	Aplicar regla de usuarios mínimos por proyecto.
Falta de adopción de negocio	La demanda sigue entrando informalmente.	Responsables de línea deben velar por HU y bugs en el proyecto correcto.
Falta de alineación con Infraestructura	CI/CD y ambientes quedan como promesa futura.	Incluir a Dayana y Jefferson en la transición técnica.
Falta de alineación QA/BA	QA llega tarde y BA no estructura historias.	Socializar modelo con Jenny y equipo antes del hito operativo.

7.7 Brechas críticas por prioridad

Prioridad	Brecha	Acción requerida
Alta	Crear proyectos Azure DevOps.	Sesiones Jefferson + líderes de frente.
Alta	Definir usuarios, roles y permisos.	Aplicar mínimo privilegio por proyecto.
Alta	Cargar backlog inicial.	Reuniones con responsables de línea y apoyo de Alexandra.
Alta	Alinear QA / BA.	Socialización con Jenny y equipo.
Alta	Alinear Infraestructura / SRE.	Confirmar ambientes, secretos, conexiones y pipelines mínimos.
Media	Definir métricas ejecutivas.	Dashboard de backlog, WIP, velocidad y calidad.
Media	Reportes.	Identificar backlog de reportería y participación de José.
Media	SonarQube.	Definir umbrales iniciales y repositorios priorizados.

7.8 Lectura ejecutiva del estado actual

Ya existe el diagnóstico.
Ya existe el modelo objetivo.
Ya existe la base documental.
Ya empezó la transición con negocio.
Ahora debe materializarse en Azure DevOps mediante Humano-SL, Fomag-SL y Paralelos-SL.

8. Primera Ola de Adopción y Plan de Transición

8.1 Propósito del plan de transición

El plan de transición debe definir cómo se pasa del modelo actual al modelo objetivo sin detener la operación y sin dejar que convivan indefinidamente dos formas distintas de trabajo.

La primera ola se concentra en la creación, configuración y adopción de los proyectos Azure DevOps:

Humano-SL
Fomag-SL
Paralelos-SL

8.2 Principio rector de la transición

La transición no consiste únicamente en crear tableros. Consiste en cambiar la forma en que se gobierna la demanda.

Toda Historia de Usuario y todo bug productivo debe tener proyecto, tablero, responsable, prioridad

8.3 Hito operativo

El hito operativo definido para esta primera ola es:

1 de julio de 2026

A partir de esa fecha, las nuevas Historias de Usuario y bugs productivos deberán gestionarse en Azure DevOps dentro del proyecto y tablero correspondiente.

8.4 Alcance de la primera ola

Elemento	Alcance
Proyectos Azure DevOps	Humano-SL, Fomag-SL y Paralelos-SL.
Tableros por proyecto	Board Desarrollo y Board Soporte.
Usuarios	Solo usuarios reales del proyecto, con roles y permisos mínimos.
Backlog inicial	HU y bugs productivos levantados con responsables de línea de negocio.
Configuración técnica	SRE / Cloud con líderes de frente.
Calidad	QA integrado al flujo, sin Board QA independiente.
Reportes	Participación de José cuando el backlog incluya reportería.
Apoyo operativo	Alexandra apoya carga, organización y completitud inicial.

8.5 Secuencia de adopción inicial

Paso	Acción	Responsable principal	Apoyo
1	Reuniones con responsables de línea de negocio.	Jonathan Moreno / líderes de frente	Alexandra Castillo
2	Levantamiento inicial de HU y bugs productivos.	Responsables de línea de negocio	BA / Alexandra / líderes de frente
3	Definición del proyecto destino: Humano-SL, Fomag-SL o Paralelos-SL.	Líderes de frente	Alexandra
4	Definición del tablero destino: Desarrollo o Soporte.	Líderes de frente	Alexandra / QA / Soporte
5	Configuración del proyecto Azure DevOps.	Jefferson Contreras	Líderes de frente
6	Carga de usuarios reales, roles y permisos mínimos.	Jefferson Contreras	Líderes de frente

Paso	Acción	Responsable principal	Apoyo
7	Carga inicial del backlog.	Responsables de negocio / líderes de frente	Alexandra
8	Revisión de completitud mínima.	Líderes de frente	Alexandra / QA / BA
9	Activación operativa del tablero.	Líderes de frente	Jefferson
10	Seguimiento inicial de adopción.	Jonathan Moreno	Gerencia técnica / líderes de frente

8.6 Configuración mínima de cada proyecto

Cada proyecto Azure DevOps deberá cumplir como mínimo:

- Proyecto configurado como privado.
- Proceso CMMI.
- Control de versiones Git.
- Team Desarrollo.
- Team Soporte.
- Sin Team QA ni Board QA independiente.
- Board Desarrollo configurado.
- Board Soporte configurado.
- Usuarios reales del frente.
- Roles y permisos mínimos.
- Area Paths e Iteration Paths definidos.
- Backlog inicial cargado o preparado.
- Ambientes Dev, SIT, UAT, Staging y Prod identificados cuando aplique.
- Repositorios definidos cuando aplique.
- Branch policies y PR obligatorios cuando exista código.
- SonarQube / Quality Gate definido cuando aplique.
- Evidencias iniciales de configuración.

8.7 Reglas inmediatas

Regla	Propósito
Ningún trabajo relevante debe quedar solo en chat, correo o reunión.	Evitar trabajo invisible.
Toda HU debe ir al proyecto y tablero correcto.	Medir demanda y capacidad.
Todo bug productivo debe ir al Board Soporte correspondiente.	Ordenar atención operativa y evidencia.
Cada proyecto solo debe tener usuarios reales del frente.	Controlar permisos y métricas.
QA se integra al flujo, no se crea Board QA.	Evitar silos de calidad.
SRE configura con líderes, no de forma aislada.	Asegurar que el proyecto nazca útil y gobernado.
Alexandra apoya carga y organización, no reemplaza BA.	Evitar confusión de responsabilidades.
José participa cuando exista reportería.	Usar apoyo especializado sin crear un frente paralelo.

8.8 Decisiones pendientes

Decisión	Responsable sugerido	Observación
Confirmar usuarios por proyecto.	Líderes de frente + Jefferson.	Debe aplicar mínimo privilegio.
Confirmar responsables funcionales por línea.	Gerencia / negocio.	Base para backlog inicial.
Confirmar flujo de bugs productivos.	Líderes Soporte + QA.	Debe ir a Board Soporte.
Confirmar participación de QA/BA.	Jenny Quintana.	QA integrado, BDD y criterios verificables.
Confirmar participación de Infraestructura.	Dayana González.	Ambientes, secretos y despliegues.
Confirmar tableros y Area Paths.	Jefferson + Líderes de frente.	Humano-SL, Fomag-SL y Paralelos-SL.
Confirmar hito de activación.	Gerencia técnica.	1 de julio de 2026.

8.9 Entregables esperados del plan de transición

Entregable	Resultado esperado
Proyectos Azure DevOps creados.	Humano-SL, Fomag-SL y Paralelos-SL listos para operar.
Tableros configurados.	Board Desarrollo y Board Soporte por proyecto.
Usuarios y permisos asignados.	Solo usuarios reales con mínimo privilegio.
Backlog inicial cargado.	HU y bugs productivos visibles.
Responsables de línea identificados.	Dueños funcionales de demanda.
Reglas de operación publicadas.	Cero trabajo invisible.
Evidencia de configuración.	Auditoría inicial de adopción.
Dashboard inicial.	Lectura de backlog, WIP, capacidad y bloqueos.

8.10 Criterios de éxito de la transición

La transición se considerará exitosa cuando:

- Humano-SL, Fomag-SL y Paralelos-SL estén creados en Azure DevOps.
- Cada proyecto tenga Board Desarrollo y Board Soporte.
- Cada proyecto tenga solo los usuarios requeridos.
- Las HU y bugs productivos estén cargados en el proyecto correcto.
- Los responsables de negocio conozcan su responsabilidad.
- QA/BA e Infraestructura estén articulados al flujo.
- SRE haya aplicado el checklist de configuración.
- El trabajo nuevo deje de entrar por canales informales.
- Existan métricas iniciales de backlog, WIP y capacidad.

8.11 Riesgos de transición y mitigación

Riesgo	Mitigación
El tablero se crea pero no se usa.	Hito del 1 de julio y regla de no iniciar trabajo sin Work Item.
Las HU entran incompletas.	Revisión mínima con responsables de negocio, BA y Alexandra.
Los bugs productivos entran como desarrollo.	Separar Board Soporte y clasificar correctamente.
Se cargan demasiados usuarios.	Regla de usuarios mínimos por proyecto.
QA queda por fuera del flujo.	Integrar QA en Ready for QA / In QA dentro de cada tablero.

Riesgo	Mitigación
Infraestructura recibe despliegues sin evidencia.	Exigir artefactos, aprobaciones, Work Items y pipelines.
Reportes se tratan como solicitudes sueltas. SRE configura sin líderes.	Involucrar a José cuando exista reportería. Sesiones obligatorias Jefferson + líderes del frente.

8.12 Cierre ejecutivo

El TO-BE ya tiene base conceptual, documental y operativa. El siguiente paso no es seguir documentando indefinidamente, sino materializar la primera ola en Azure DevOps.

El cambio esperado es claro:

Antes:

La demanda entraba por múltiples canales, con baja trazabilidad y responsables difusos.

Ahora:

Cada necesidad debe entrar al proyecto correcto, al tablero correcto, con responsable, prioridad, evidencia y capacidad medible.

La primera ola debe demostrar que la compañía puede operar como fábrica de software gobernada, empezando por Desarrollo y Arquitectura y articulando de forma ordenada a QA/BA, Infraestructura, negocio, SRE, proveedores y gerencia.

Ajuste complementario — Gobierno de demanda y refinamiento anticipado del backlog

A. Punto de partida ajustado

El TO-BE reconoce que la organización ya utiliza Azure DevOps y que Jenny ya cumple un rol relevante de BA / QA antes del ciclo. Por tanto, el modelo objetivo no busca introducir desde cero la participación funcional ni la herramienta, sino madurar su uso y formalizar reglas de gobierno más consistentes.

La brecha a resolver es:

Demanda distribuida por múltiples líneas.

Posibles historias duplicadas o solapadas.

Dependencia manual de refinamiento.

Ciclos largos con cambios de prioridad.

Incendios operativos que desplazan trabajo planificado.

Historias que pueden quedar parcialmente ejecutadas o sin validación completa.

B. Regla TO-BE de refinamiento anticipado a 2-3 sprints

Todo Requirement candidato a Sprint debe pasar por una mesa de refinamiento anticipado con participación mínima de:

- Jefe de línea de negocio.
- Jenny en rol BA / QA.
- Líder técnico del frente.

Esta preparación debe realizarse idealmente con 2 a 3 sprints de anticipación frente al sprint en el que se espera ejecutar el trabajo.

C. Objetivo del refinamiento anticipado

El objetivo es asegurar que cada historia llegue al Sprint Planning con:

- Necesidad clara.
- Valor de negocio explícito.
- Prioridad validada por la línea.
- Duplicidades revisadas.
- Alcance suficientemente cerrado.
- Criterios de aceptación verificables.
- Escenarios BDD cuando apliquen.
- Dependencias identificadas.
- Riesgos técnicos visibles.
- Estimación preliminar.
- Responsable funcional de aceptación.

D. Flujo objetivo de preparación

Demanda cargada por línea de negocio

↓

Revisión de duplicidad y consolidación

↓

Refinamiento con jefe de línea + BA/QA + líder técnico

↓

Validación de prioridad, alcance, criterios y dependencias

↓

Ready for Sprint

↓

Sprint Planning

↓

Sprint Backlog

E. Responsabilidades por rol

Rol	Responsabilidad
Jefe de línea de negocio	Define necesidad, valor, prioridad, impacto y alcance funcional. Valida duplicidades con solicitudes existentes y confirma responsable de aceptación.
Jenny — BA / QA	Estructura la HU, depura duplicidades, fortalece criterios de aceptación, redacta o valida BDD, asegura calidad funcional y trazabilidad mínima.
Líder técnico	Evalúa factibilidad, dependencias, riesgos, impacto sobre componentes, esfuerzo estimado y restricciones técnicas.
Equipo Scrum	Revisa capacidad disponible y compromete trabajo únicamente durante el Sprint Planning.

F. Regla de Sprint Planning

El Sprint Planning no debe utilizarse para descubrir, depurar o reconstruir historias inmaduras.

Solo pueden entrar al Sprint Planning historias que cumplan Definition of Ready.

Las urgencias, incendios o bugs productivos deben gestionarse mediante el flujo de Soporte / Kanban o mediante una regla explícita de interrupción controlada, evitando contaminar la lectura del compromiso del sprint.

G. Resultado esperado

El T0-BE protege el sprint corto mediante preparación anticipada.
BA / QA no trabaja contra el reloj.
El líder técnico detecta riesgos antes del compromiso.
El jefe de línea valida valor y prioridad.
El equipo compromete menos incertidumbre y más trabajo realmente ejecutable.

Ajuste complementario — Frentes operacionales, Jefferson, Alexandra y límites de responsabilidad

1. Claridad organizacional sobre Jefferson Contreras

Jefferson Contreras pertenece al área de **Desarrollo y Arquitectura**, bajo el liderazgo de Jonathan Moreno, y opera como apoyo transversal técnico para:

- Azure DevOps.
- SRE / Cloud.
- CI-CD.
- SonarQube.
- Gobierno técnico.
- Onboarding técnico de proyectos y repositorios.
- Evidencias técnicas de configuración.

Su rol es transversal porque apoya a Humano-SL, Fomag-SL y Paralelos-SL, pero no constituye un frente operacional independiente ni reemplaza al líder técnico de cada frente.

Desarrollo y Arquitectura

Líder: Jonathan Moreno

- └─ Humano-SL
- └─ Fomag-SL
- └─ Paralelos-SL
- └─ Apoyo transversal técnico
 - └─ Jefferson Contreras: Azure DevOps / SRE / Cloud / CI-CD / Sonar / gobierno técnico

2. Relación funcional entre Jefferson y Alexandra

Alexandra Castillo opera como apoyo documental y operativo de adopción Azure DevOps, articulada funcionalmente con Jefferson.

Esto significa que Jefferson coordina técnicamente la adopción, configuración y gobierno de Azure DevOps, mientras Alexandra apoya la carga inicial, organización, completitud y trazabilidad básica de Work Items.

Esta relación debe leerse como **coordinación funcional de trabajo**, no necesariamente como subordinación jerárquica formal, salvo que la organización lo defina explícitamente por fuera de este documento.

Jefferson coordina técnicamente la adopción Azure DevOps.

Alexandra apoya operativamente la carga y organización de información.

Jonathan Moreno conserva el liderazgo del área de Desarrollo y Arquitectura.

3. Límites del rol de Alexandra Castillo

Alexandra no reemplaza al área BA / QA ni asume responsabilidad funcional sobre las Historias de Usuario.

Sus responsabilidades esperadas son:

- Apoyar la carga inicial de Historias de Usuario y bugs productivos.
- Revisar completitud básica de campos obligatorios.
- Apoyar la organización inicial del backlog.
- Ayudar a ubicar Work Items en el proyecto y tablero correcto.
- Apoyar la trazabilidad documental dentro de Azure DevOps.
- Escalar dudas funcionales al jefe de línea de negocio y a Jenny / BA-QA.
- Escalar dudas técnicas a Jefferson o al líder técnico del frente.

Límites explícitos:

- No define criterios de aceptación de negocio.
- No prioriza el backlog.
- No reemplaza a Jenny en la función BA / QA.
- No decide si una necesidad es válida funcionalmente.
- No aprueba calidad funcional.
- No asume responsabilidad por la estimación técnica.

4. Jefferson y la calidad técnica del código

Jefferson habilita y monitorea mecanismos objetivos de calidad técnica, principalmente:

- Configuración de SonarQube / SonarCloud.
- Quality Gates.
- Integración de análisis estático en pipelines.
- Evidencias de validación técnica.
- Reportes de deuda técnica, bugs, vulnerabilidades y mantenibilidad.
- Gobierno base de pipelines y políticas automatizadas.

Sin embargo, Jefferson no reemplaza la responsabilidad técnica del desarrollador ni la revisión del líder técnico del frente.

La calidad del código implementado es responsabilidad primaria del desarrollador.

La revisión técnica y aprobación del cambio es responsabilidad del líder técnico o reviewer de

Jefferson habilita los controles automatizados y evidencia objetiva de calidad.

SonarQube complementa, pero no reemplaza, el Pull Request ni el code review.

5. Matriz ajustada de responsabilidad

Tema	Responsable primario	Apoyo / gobierno	Límite explícito
Gobierno del frente Humano-SL	Líder Humano-SL	Jefferson / Alexandra según necesidad	Jefferson no reemplaza al líder del frente.
Gobierno del frente Fomag-SL	Líder Fomag-SL	Jefferson / Alexandra según necesidad	Jefferson no prioriza negocio.
Gobierno del frente Paralelos-SL	Líder Paralelos-SL	Jefferson / Alexandra según necesidad	Jefferson no asume liderazgo funcional del frente.
Configuración Azure DevOps	Jefferson	Líder del frente / Alexandra	No se configura sin usuarios, roles, tableros y alcance definidos.
Carga inicial de HU y bugs	Jefe de línea / BA / responsables funcionales	Alexandra	Alexandra apoya carga y orden; no define contenido funcional.
Refinamiento funcional	Jenny / BA-QA + jefe de línea	Líder técnico	Alexandra no reemplaza el refinamiento BA.

Tema	Responsable primario	Apoyo / gobierno	Límite explícito
Priorización de negocio	Jefe de línea / PO / negocio	Jenny / BA-QA	Jefferson y Alexandra no priorizan valor de negocio.
Estimación técnica	Líder técnico + equipo de desarrollo	Jenny / BA-QA para contexto funcional	Jefferson no estima en nombre del equipo salvo alcance técnico de plataforma.
Calidad estática	Jefferson	Líder técnico / desarrolladores	Sonar mide y bloquea; no reemplaza code review.
Calidad del código	Desarrollador + líder técnico	Jefferson mediante controles automatizados	La responsabilidad técnica del código no se traslada a SRE.
Calidad funcional	Jenny / QA-BA	Jefe de línea / QA / Desarrollo	QA valida contra criterios; no define arquitectura.
CI/CD y pipelines	Jefferson + Infraestructura según alcance	Líder técnico / responsables de ambiente	Jefferson no reemplaza la responsabilidad operativa de Infraestructura.

6. Lectura correcta del modelo

Jefferson pertenece a Desarrollo y Arquitectura y gobierna técnicamente la adopción Azure DevOps.
 Alexandra pertenece al apoyo operativo de adopción Azure DevOps y trabaja articulada funcionalmente.
 Jenny conserva la responsabilidad BA / QA funcional.
 Los líderes técnicos conservan la responsabilidad sobre revisión técnica y calidad del código.
 Los jefes de línea conservan la responsabilidad sobre necesidad, prioridad y valor de negocio.

7. Riesgos que evita esta precisión

Riesgo	Cómo lo evita el ajuste
Pensar que Alexandra reemplaza a BA.	Se define como apoyo operativo, no funcional.
Pensar que Jefferson es jefe formal de Alexandra sin validación organizacional.	Se habla de coordinación funcional, no de jerarquía formal.
Trasladar la calidad del código a SRE.	Se conserva responsabilidad en desarrollador y líder técnico.
Reemplazar code review con Sonar.	Se aclara que Sonar complementa, no sustituye, el Pull Request.
Mezclar SRE con Infraestructura.	Se define apoyo técnico transversal, articulado con Infraestructura según alcance.
Debilitar responsabilidad de los líderes de frente.	Se aclara que Jefferson apoya, pero no reemplaza liderazgo operativo.

Ajuste complementario final — Primera ola de migración de HU y clasificación de demanda para jefes de línea

1. Propósito

Este ajuste precisa la tercera etapa de adopción y corrige la lectura operativa de la migración de Historias de Usuario. La primera ola no termina con tableros configurados ni con socialización. El paso siguiente es la capacitación práctica y carga asistida de demanda con Alexandra, conservando los límites de responsabilidad de negocio, BA/QA, líderes técnicos y SRE.

2. Ruta real de adopción de la primera ola

La secuencia correcta de implementación debe quedar así:

- Paso 1. Configuración de tableros y estructura base.
- Paso 2. Socialización del plan con cada jefe de línea de negocio.
- Paso 3. Capacitación y carga asistida de Historias de Usuario, bugs y casos.
- Paso 4. Revisión de completitud mínima y ubicación correcta.
- Paso 5. Refinamiento funcional con jefe de línea + Jenny BA/QA + líder técnico.
- Paso 6. Validación técnica y preparación del backlog candidato.
- Paso 7. Sprint Planning sobre historias Ready.

3. Paso 3 corregido — Capacitación y carga asistida con Alexandra

Alexandra participa como apoyo operativo de adopción Azure DevOps. Su función en esta etapa es enseñar, acompañar y verificar que los jefes de línea o sus responsables designados carguen la demanda correctamente.

No debe interpretarse que Alexandra migra sola la demanda, prioriza historias, define criterios de aceptación o reemplaza a Jenny / BA-QA.

Actividad	Responsable	Límite
Capacitar en uso básico de Azure DevOps.	Alexandra	No define prioridad ni alcance funcional.
Acompañar la carga inicial de HU, bugs y casos.	Alexandra + responsables designados por línea	La necesidad sigue siendo responsabilidad de negocio.
Revisar campos mínimos.	Alexandra	No reemplaza refinamiento BA.
Ayudar a ubicar proyecto y tablero.	Alexandra	La clasificación puede ser ajustada por líderes/BA durante revisión.
Resolver dudas de estructura Azure DevOps.	Alexandra / Jefferson	Las dudas funcionales pasan a jefe de línea y Jenny.

4. Responsabilidad de los jefes de línea

Los jefes de línea no solo reciben socialización. Deben asegurar que la demanda de su frente quede registrada con contexto suficiente.

El jefe de línea responde por necesidad, prioridad, impacto, contexto funcional y responsable. Alexandra acompaña la carga operativa. Jenny refina funcionalmente. El líder técnico valida factibilidad.

5. Guía de clasificación para negocio

Para facilitar la adopción, la clasificación se debe explicar en lenguaje no técnico.

Desarrollo: cuando se necesita que el sistema haga algo nuevo, mejorado o diferente de forma p
 Soporte: cuando algo que ya existe falla, bloquea la operación, requiere diagnóstico o necesit

Situación observada por negocio	Debe ir a	Razón
Nueva funcionalidad.	Board Desarrollo	Construye nueva capacidad.
Mejora funcional planificada.	Board Desarrollo	Cambia o mejora capacidad existente.
Nuevo reporte.	Board Desarrollo	Crea una capacidad nueva.
Nuevo filtro o nueva regla de negocio futura.	Board Desarrollo	Es evolución planificada.
Error en producción.	Board Soporte	Afecta operación actual.
Usuario bloqueado o sin acceso.	Board Soporte	Es atención operativa.
Reporte existente falla o muestra datos incorrectos.	Board Soporte	Requiere diagnóstico/corrección.
Cálculo actual está errado en un proceso ya operativo.	Board Soporte inicialmente	Se diagnostica; puede derivar en bug de código.
Urgencia o incendio productivo.	Board Soporte	Requiere estabilización.
Solicitud de liberación o despliegue aprobado.	Board Soporte / Release	Es operación de promoción.

6. Regla de casos grises

Si negocio no sabe si es Desarrollo o Soporte, debe cargar el caso con el mayor contexto posib
 Si el caso describe falla, afectación actual o incertidumbre operativa, entra primero por Sopo
 Si después del diagnóstico se confirma que es mejora o nueva capacidad, se deriva a Desarrollo

Esta regla evita que los jefes de línea se bloqueen por clasificación técnica. La prioridad es capturar buena información y ajustar la clasificación durante revisión.

7. Campos mínimos para carga inicial

Tipo de demanda	Información mínima
Historia de Usuario / mejora	Línea solicitante, producto/módulo, necesidad, valor esperado, prioridad, responsable funcional, evidencia o ejemplo.
Bug productivo / incidente	Ambiente, síntoma, usuario/proceso afectado, evidencia, impacto, urgencia y pasos de reproducción si existen.
Reporte	Código o nombre funcional del reporte si existe, objetivo, filtros esperados, fuente funcional de datos, salida esperada y responsable de validación.
Acceso / permiso	Usuario afectado, rol esperado, ambiente, proceso bloqueado y autorización funcional.

8. Control de duplicidades durante migración

Como varias líneas pueden impactar el mismo producto, especialmente Humano, la carga inicial debe incluir revisión de duplicidades.

Antes de considerar una HU candidata a Sprint, Jenny / BA-QA y el jefe de línea deben validar

La duplicidad no siempre implica eliminar una historia. Puede implicar consolidar alcance, relacionar Work Items, separar por cliente o definir una Feature común.

9. Estado de una HU cargada

Una HU cargada durante la migración no queda automáticamente lista para construir.

Cargada no significa refinada.

Refinada no significa comprometida.

Comprometida no significa terminada.

Terminada exige validación y evidencia.

10. Resultado esperado

Con este ajuste, la primera ola permite:

- Enseñar a negocio cómo cargar demanda.
- Reducir errores de clasificación.
- Evitar que Alexandra asuma responsabilidades BA.
- Dar claridad a los jefes de línea.
- Separar carga inicial de compromiso de Sprint.
- Proteger el refinamiento funcional y técnico.
- Mejorar trazabilidad desde el origen de la demanda.